Probit Regression Function

Applications in Government & Politics

Elliot Hipski, Rohan, Brandon, Arnav

April 2026

Table of Contents

1. Prerequisite Knowledge
2. General Explanation
3. Mathematical Explanation
4. Implementation
5. Comparison to Other Models
6. Citations and Appendix

Topic 1 | Prerequisite Knowledge

Bernoulli Distribution Variance

Used for modeling binary outcomes. The variance is defined as $\text{Var}(X) = E[X^2] - (E[X])^2 = p - p^2 = p(1-p)$

General Linearized Model

A *General Linearized Model* is a model that applies a linear function to a dataset, with the general rules of mapping the input to the output y, and mapping it according to a **Link Function**.

Gradient

A vector of partial derivatives. It is used in mapping the log likelihood function of the linear model. For this case, we use the weights as slopes and apply the transformation $X^T \cdot W$, where W is the weights matrix. This is used in solving for the **Hessian**

Topic 1 | Prerequisite Knowledge

Hessian

A matrix of 2nd order partial derivatives. For this case, however, we take the negative Expected Value of the hessian, which simplifies the matrix math. Using the previously defined weights matrix, a formula for solving the Expected Hessian becomes (2):

$$X^T W X^T$$

Link Function

Topic 2 | General Explanation

- **Assumptions**:
  - Has all assumptions of a GLM, found in (4)
  - Assumes a Bernouli Distribution for the y data
- **Inputs**:
  - x: the design matrix, defined as having all 1s in the first column vector, with the rest being vectors for the observed values of the data
  - y: the target variable, assumed Bernouli Distribution
  - max_iter: maximum iterations to do
  - tolerance: the level that is closest to zero that decides if there has been a "convergence" to the $\hat{B}$
  - verbose: used for debugging, prints the steps and beta values
  - *note* some of these inputs are handled by glm differently, like the design matrix X is created by just adding a column of 1s to the dataframe passed in as x

Topic 2 | General Explanation

- **Function**:
  - initialize beta to be a vector of 0s, of length m (number of x columns)
  - for(i in max_iter)
    - get the linear predictor, eta
    - calculate the PDF and CDF of the predictor assuming the normal distribution, defined as $\hat{p}\hat{q}$
    - calculate the ratio of the CDF to the bernouli variance of the PDF $\hat{q}/(\hat{p}(1-\hat{p}))$
    - get a score vector with the previous ratio in the form ratio $\cdot\,(\hat{y}-\hat{p})$
    - calculate the gradient, g, defined by $X^T \cdot$ score
    - now, we calculate the Weight matrix of the form $\frac{\hat{q}}{\hat{p}(1-\hat{p})}$ note the bernouli distribution in the denominator
    - using the Weight vector, we calculate the negative expected Hessian, H, this is written as $-(X^T \cdot W)$
    - finally, a copy of the current beta iteration is made, and the next iteration of beta becomes $\hat{B} = \hat{B} - (H^{-1}g)$ where g is the gradient
    - then the tolerance check is done, checking the squared sum of $\hat{B}_{new} - \hat{B}_{old}$, or $||B_{new} - B_{old}||^2$ if this is less than our tolerance, we have converged

Topic 2 | General Explanation

- **Usage**:
  - Probit has many uses, but most of them are in econometrics and Government. The closest model in terms of use case is Logistic, as they both follow bernouli variables.
  - The main difference from Logisitc regression is that logistic regression uses the logistic distribution
  - This makes it useful in studying things like human behavior, psychology, economics, and finance.

Topic 3 | Mathematical Explanation

Topic 4 | Implementation

```r
myprobit = function(X, y, max_iter, tolerance, verbose=FALSE){

  m = ncol(X)
  beta <- rep(0, m)

  for (i in 1:max_iter) {

    eta <- X %*% beta

    phat <- pnorm(eta)
    qhat <- dnorm(eta)

    ratio <- qhat / (phat * (1 - phat))
    score <- (y - phat) * ratio
    g <- t(X) %*% score

    w <- (qhat^2) / (phat * (1 - phat))
    H <- -t(X) %*% (X * as.vector(w))

    copy_beta = beta
    beta <- beta - solve(H, g)

    if (verbose){
      cat("Iteration", i, ":\n")
      j=1
      for (b in beta){
        cat("Beta", j, " = ", beta[j], "\n")
        j=j+1
      }
    }
    if ((sum(beta - copy_beta)^2) < tolerance) {
        return(beta)
        break
      }
  }
}
```

Topic 4 | Implementation

```r
X <- model.matrix(am ~ mpg, data = mtcars)
y <- mtcars$am
myprobit(X,y,32, 0.00009, FALSE)
```

```
##                    [,1]
## (Intercept) -3.9100663
## mpg          0.1822605
```

Topic 4 | Implementation

Solving the System

Optimization Methods

- There are 2 main methods of creating a model for probit regression:
  - 1. Newton-Raphson method We used a method similar to Newton Raphson, but rather than use the Hessian, the Expected Hessian was used, which is used for IRLS and is simpler to calculate.
  - 2. Iteratively Reweighted Least Squares / Fisher Scoring
- These methods can be shown to be mathematically the same(7)

Testing

Topic 5 | Comparison to Other Models

Motivation

Several methods exist for modeling binary outcomes. This section compares probit regression against three alternatives:

1. **Linear Regression (OLS)** — the standard baseline for continuous outcomes
2. **Logistic Regression** — the most widely used GLM for binary data
3. **Lasso Regression** — a penalized method that performs automatic variable selection

The goal is to understand the practical and theoretical differences between these approaches, and to identify when probit regression is the appropriate choice.

Data: Donner Party Survival

We use the **Donner Party survival dataset** from the `Sleuth3` package. The binary outcome is survival status (1 = Survived, 0 = Died), with Age and Sex as predictors.

```r
library(Sleuth3); library(dplyr); library(ggplot2)
library(glmnet);  library(caret)

set.seed(42)

# recode response and sex
donner <- case2001 %>%
  mutate(
    mystatus = if_else(Status == "Survived", 1, 0),
    male     = if_else(Sex == "Male", 1, 0)
  )

head(donner[, c("Age", "Sex", "Status", "mystatus")], 4)
```

```
##   Age    Sex   Status mystatus
```

Linear Regression on a Binary Response

OLS fits:

$$\hat{E}(Y \mid \mathbf{X}) = \hat{\beta}_0 + \hat{\beta}_1 X_1 + \cdots + \hat{\beta}_p X_p$$

When $Y \in \{0, 1\}$, this is the **Linear Probability Model (LPM)**. Since $\mathbf{X} \in \mathbb{R}$, fitted values are not constrained to $[0, 1]$, which means the model can produce predicted probabilities below 0 or above 1. This violates the basic requirement for a valid probability model and is the primary motivation for using a link function in GLMs.

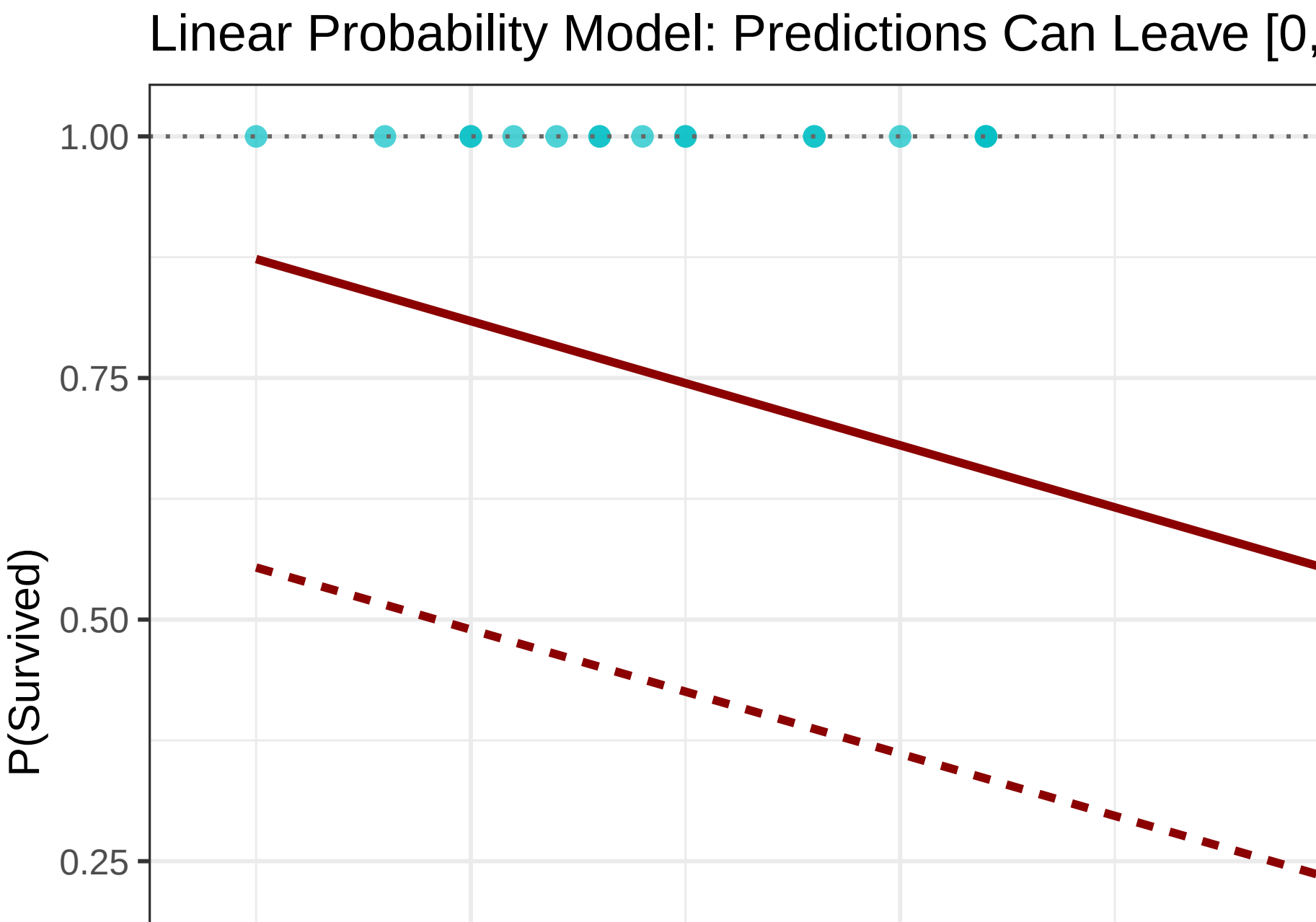
OLS on a Binary Response — Code

```r
# fit OLS
lpm_fit <- lm(mystatus ~ Age + male, data = donner)

donner <- donner %>%
  mutate(pi_lpm = predict(lpm_fit))

ggplot(donner, aes(x = Age, y = mystatus)) +
  geom_point(aes(color = Status), size = 2, alpha = 0.7) +
  geom_line(aes(y = pi_lpm, group = male,
                linetype = factor(male, labels = c("Female"
            color = "darkred", linewidth = 1) +
  geom_hline(yintercept = c(0, 1), linetype = "dotted", col
  labs(title = "Linear Probability Model: Predictions Can I
       y = "P(Survived)", x = "Age", linetype = "Sex",
       caption = "Dotted lines mark the valid [0,1] probabi
  theme_bw()
```

OLS on a Binary Response — Plot



OLS Predictions Outside [0, 1]

```r
lpm_preds <- predict(lpm_fit)

cat("Range of OLS predictions:", round(range(lpm_preds), 4)
```

```
## Range of OLS predictions: -0.0885 0.8731
```

```r
cat("Predictions below 0:     ", sum(lpm_preds < 0), "\n")
```

```
## Predictions below 0:      3
```

```r
cat("Predictions above 1:     ", sum(lpm_preds > 1), "\n")
```

```
## Predictions above 1:      0
```

```r
# misclassification at 0.5 threshold
lpm_class <- ifelse(lpm_preds > 0.5, 1, 0)
cat("OLS misclassification rate:",
    round(mean(lpm_class != donner$mystatus), 4), "\n")
```

Logistic Regression

Logistic regression uses the **logit link**:

$$\eta = \text{logit}(\pi) = \ln\left[\frac{\pi}{1-\pi}\right] = \mathbf{x}^T\boldsymbol{\beta} \quad \Rightarrow \quad \hat{\pi} = \frac{e^\eta}{1+e^\eta}$$

This assumes errors follow a logistic distribution.
Probit regression uses the **inverse standard normal CDF**:

$$\eta = \Phi^{-1}(\pi) = \mathbf{x}^T\boldsymbol{\beta} \quad \Rightarrow \quad \hat{\pi} = \Phi(\mathbf{x}^T\hat{\boldsymbol{\beta}})$$

This assumes errors follow a standard normal distribution. Both are valid GLMs for binary outcomes — they differ only in their distributional assumption.

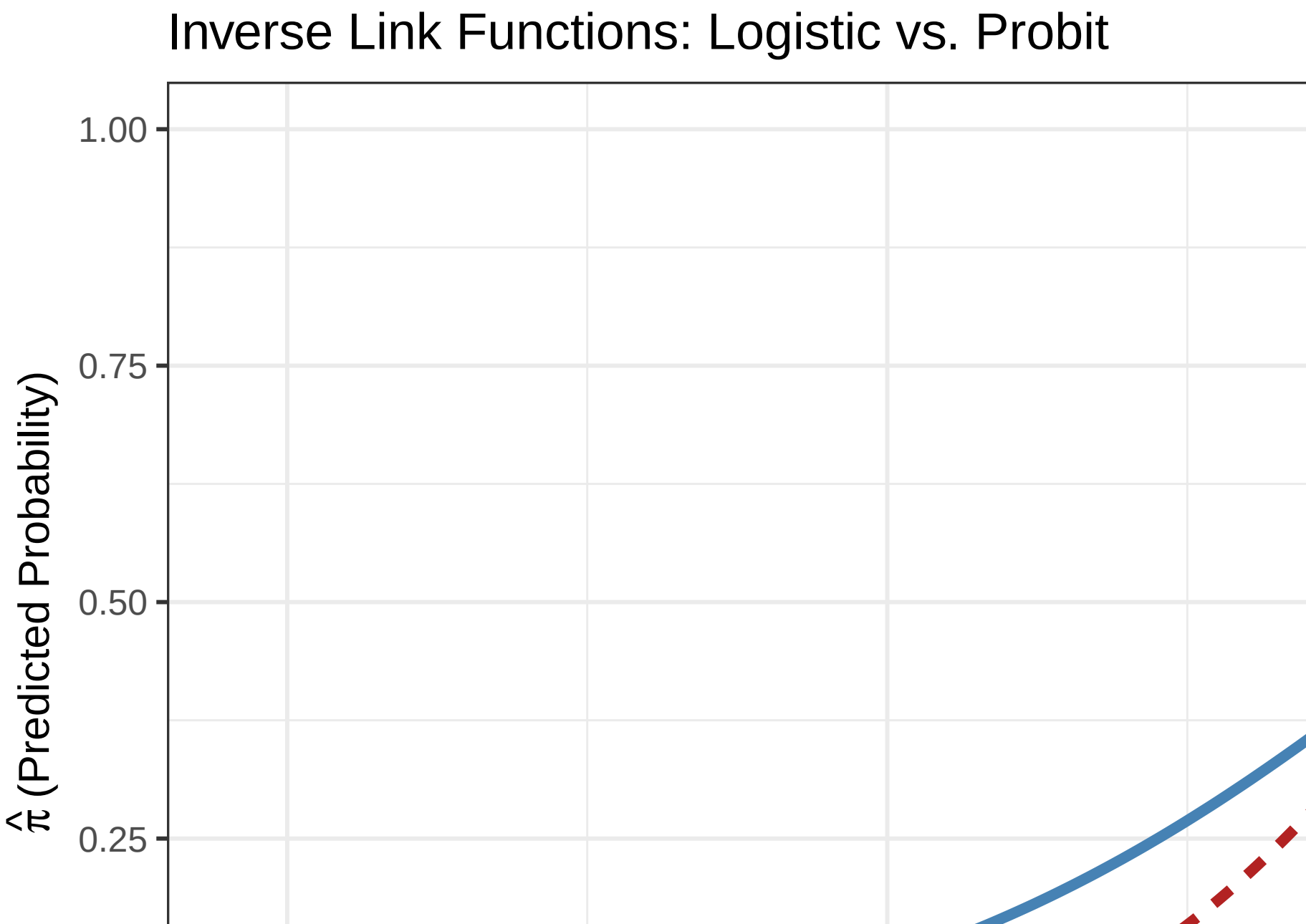
Link Functions — Code

```
eta_seq <- seq(-4, 4, length.out = 300)

df_links <- data.frame(
  eta    = rep(eta_seq, 2),
  pi     = c(plogis(eta_seq),  # inverse logit
             pnorm(eta_seq)),  # inverse probit (std norma
  Method = rep(c("Logistic (logit link)", "Probit (normal (
)

ggplot(df_links, aes(x = eta, y = pi, color = Method, linet
  geom_line(linewidth = 1.3) +
  labs(title = "Inverse Link Functions: Logistic vs. Probit
       x = expression(eta ~ "(Linear Predictor)"),
       y = expression(hat(pi) ~ "(Predicted Probability)"))
  scale_color_manual(values = c("steelblue", "firebrick"))
  theme_bw() + theme(legend.position = "bottom")
```

Link Functions — Plot



Fitting Both Models — Code

```r
# fit logistic and probit
logit_fit  <- glm(mystatus ~ Age + male, data = donner,
                  family = binomial(link = "logit"))

probit_fit <- glm(mystatus ~ Age + male, data = donner,
                  family = binomial(link = "probit"))

# compare coefficients
round(cbind(
  "OLS Coef"    = coef(lpm_fit),
  "Logit Coef"  = coef(logit_fit),
  "Probit Coef" = coef(probit_fit)
), 4)
```

Fitting Both Models — Output

```
##              OLS Coef Logit Coef Probit Coef
## (Intercept)   1.0658     3.2304      1.9173
## Age          -0.0128    -0.0782     -0.0457
## male         -0.3192    -1.5973     -0.9583
```

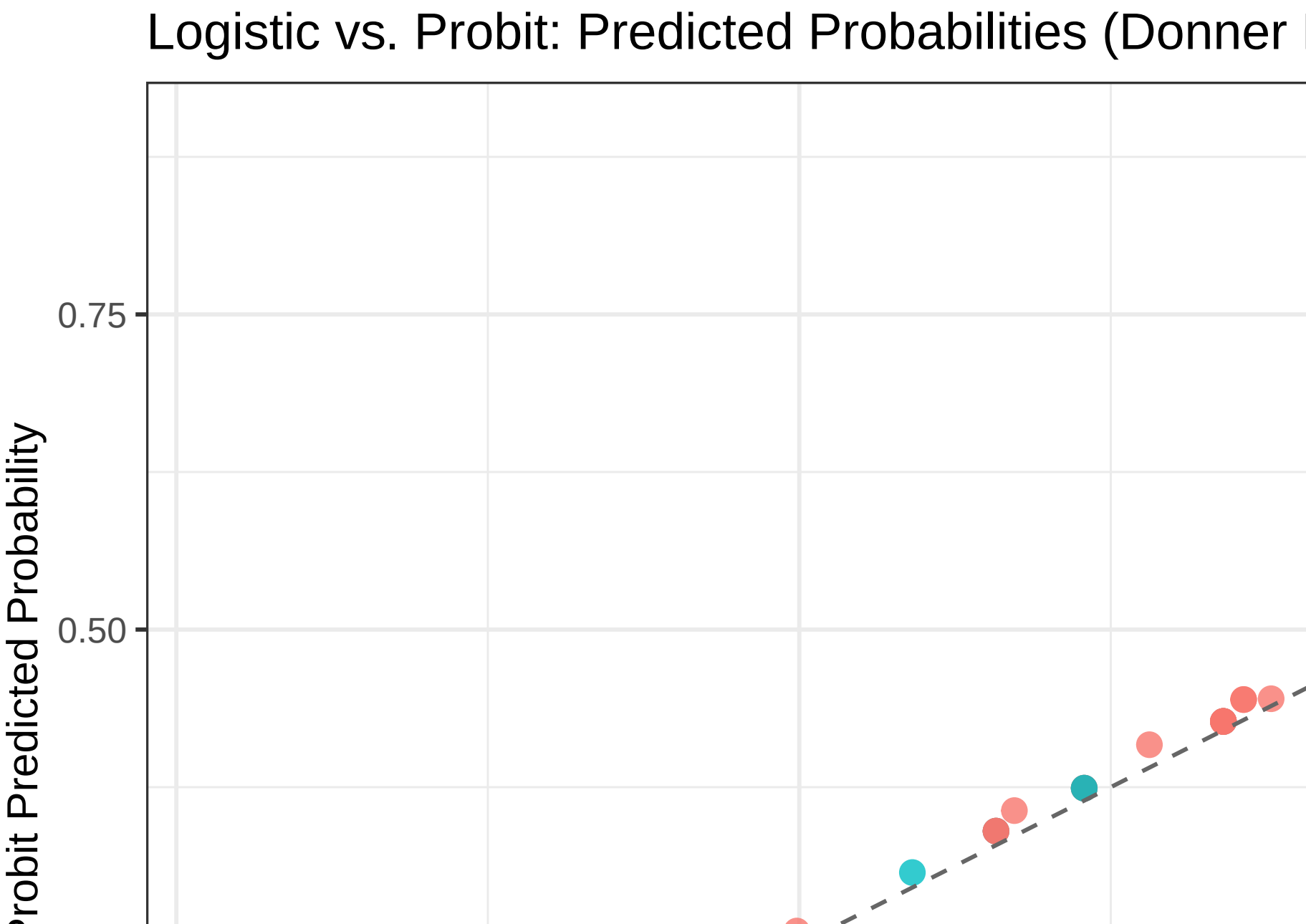
The probit coefficients are smaller by a factor of roughly 1.6 to 1.8. The logistic distribution has variance $\pi^2/3 \approx 3.29$, while the standard normal has variance 1. The scales differ, but the substantive conclusions are the same.

Predicted Probabilities — Code

```
donner <- donner %>%
  mutate(
    pi_logit  = predict(logit_fit,  type = "response"),
    pi_probit = predict(probit_fit, type = "response")
  )

ggplot(donner, aes(x = pi_logit, y = pi_probit, color = Sta
  geom_point(size = 2.5, alpha = 0.8) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed"
  labs(title = "Logistic vs. Probit: Predicted Probabilitie
       x = "Logistic Predicted Probability",
       y = "Probit Predicted Probability") +
  theme_bw()
```

Predicted Probabilities — Plot



Model Fit Metrics

```r
cat(sprintf("%-28s %8s %8s\n", "Metric", "Logistic", "Probi
```

```
## Metric                          Logistic   Probit
```

```r
cat(sprintf("%-28s %8.3f %8.3f\n", "AIC",
            AIC(logit_fit), AIC(probit_fit)))
```

```
## AIC                               57.256   57.283
```

```r
cat(sprintf("%-28s %8.3f %8.3f\n", "Residual Deviance",
            deviance(logit_fit), deviance(probit_fit)))
```

```
## Residual Deviance                 51.256   51.283
```

```r
cat(sprintf("%-28s %8.3f %8.3f\n", "Null Deviance",
            logit_fit$null.deviance, probit_fit$null.devian
```

```
## Null Deviance                     61.827   61.827
```

When to Prefer Probit

1. Latent variable framework. Assume an unobserved variable $Y^* = \mathbf{x}^T\beta + \varepsilon$ where $\varepsilon \sim N(0,1)$. We observe $Y = 1$ if $Y^* > 0$. This is the probit model exactly — a standard framework in economics and political science.

2. Tail behavior. The normal CDF approaches 0 and 1 more rapidly than the logistic distribution. Probit is preferable when probabilities near the extremes are of scientific interest.

3. Field convention. Probit is the standard choice in economics, political science, and related disciplines. For government and political science applications, probit is the conventional model.

4. Marginal effects. Probit coefficients do not have an odds-ratio interpretation, but average marginal effects provide a clear and directly interpretable quantity.

Marginal Effects

Probit coefficients have no direct probability interpretation on their own. The standard approach is to compute **Average Marginal Effects (AME)**:

$$\text{AME}_j = \frac{1}{n} \sum_{i=1}^{n} \phi(\mathbf{x}_i^T \hat{\beta}) \cdot \hat{\beta}_j$$

where $\phi(\cdot)$ is the standard normal PDF.

```r
library(margins)

# compute AMEs
ame <- margins(probit_fit)
summary(ame)[, c("factor", "AME", "SE", "z", "p")]
```

```
##  factor     AME     SE       z      p
##     Age -0.0148 0.0057 -2.6113 0.0090
##    male -0.3109 0.1188 -2.6160 0.0089
```

This gives the change in predicted probability for a one-unit change

Lasso Regression

The logistic or probit objective function can be **penalized** to perform variable selection:

$$\hat{\boldsymbol{\beta}} = \text{argmin}_{\boldsymbol{\beta}} \left\{ -\frac{2}{n}\ell(\boldsymbol{\beta}) + \lambda \sum_{j=1}^{p} |\beta_j| \right\}$$

The ℓ_1 penalty shrinks small coefficients toward zero and eliminates irrelevant predictors entirely when λ is large enough. This is particularly useful in high-dimensional settings such as political science data with many demographic and behavioral predictors.

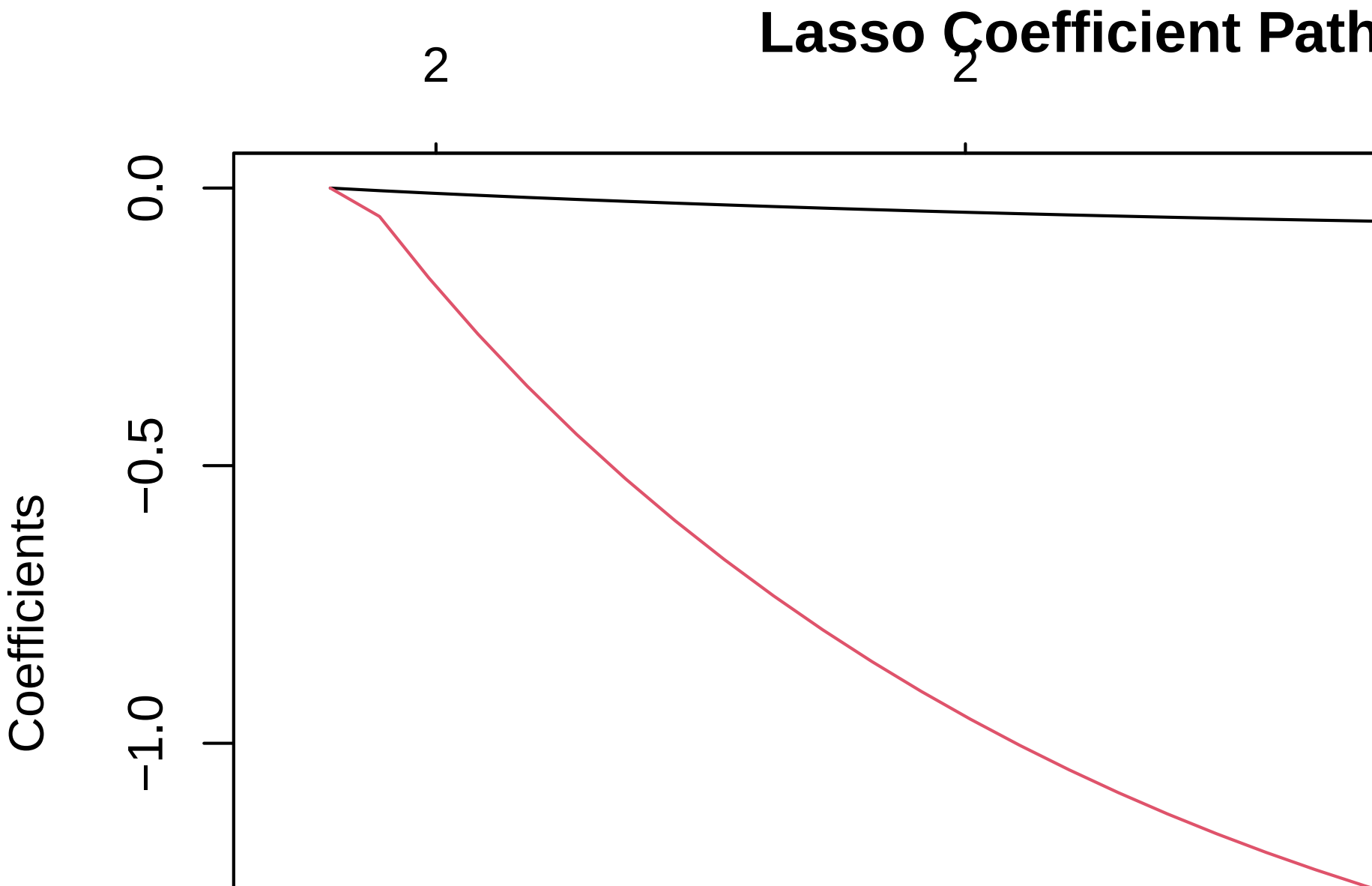
Lasso: Fitting the Model — Code

```
# glmnet needs a matrix, not a formula
X_mat <- model.matrix(mystatus ~ Age + male, data = donner)
y_vec <- donner$mystatus

# fit lasso over lambda path
lasso_path <- glmnet(X_mat, y_vec, family = "binomial", alp

plot(lasso_path, xvar = "lambda", label = TRUE,
     main = "Lasso Coefficient Path (Penalized Logistic, Do
```

Lasso: Coefficient Path



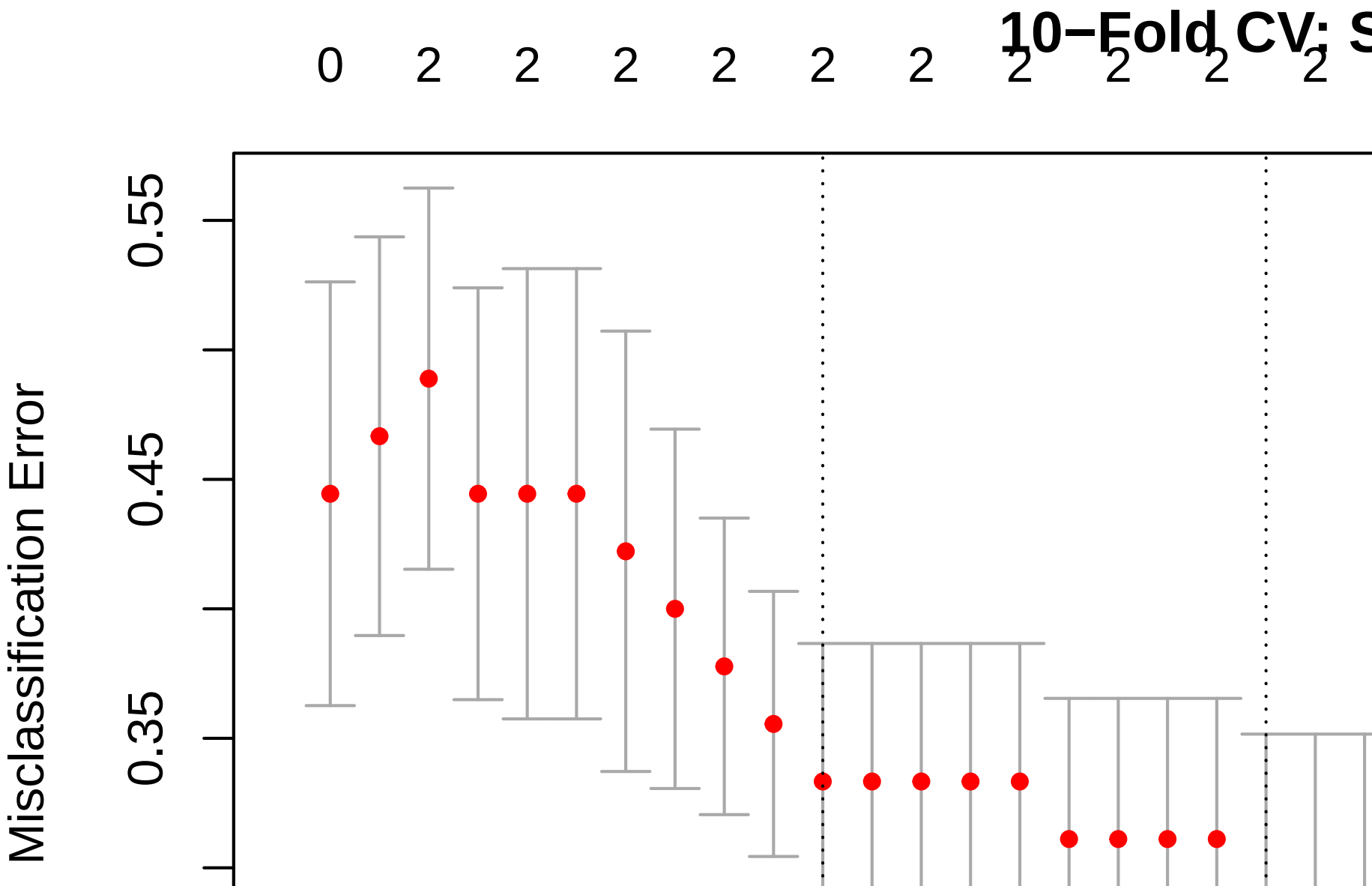
Lambda Selection via Cross-Validation — Code

```
set.seed(42)
cv_lasso <- cv.glmnet(X_mat, y_vec,
                      family       = "binomial",
                      alpha        = 1,
                      nfolds       = 10,
                      type.measure = "class")

plot(cv_lasso, main = "10-Fold CV: Selecting Optimal Lambda

cat("Lambda (min CV error): ", round(cv_lasso$lambda.min, 5
cat("Lambda (1 SE rule):    ", round(cv_lasso$lambda.1se, 5
```

Lambda Selection via Cross-Validation — Plot



Lasso Final Coefficients

```r
cat("Lasso coefficients at lambda.min:\n")
```

```
## Lasso coefficients at lambda.min:
```

```r
round(as.matrix(coef(cv_lasso, s = "lambda.min")), 5)
```

```
##              lambda.min
## (Intercept)    2.30441
## Age           -0.05615
## male          -1.19653
```

```r
cat("\nLasso coefficients at lambda.1se:\n")
```

```
##
## Lasso coefficients at lambda.1se:
```

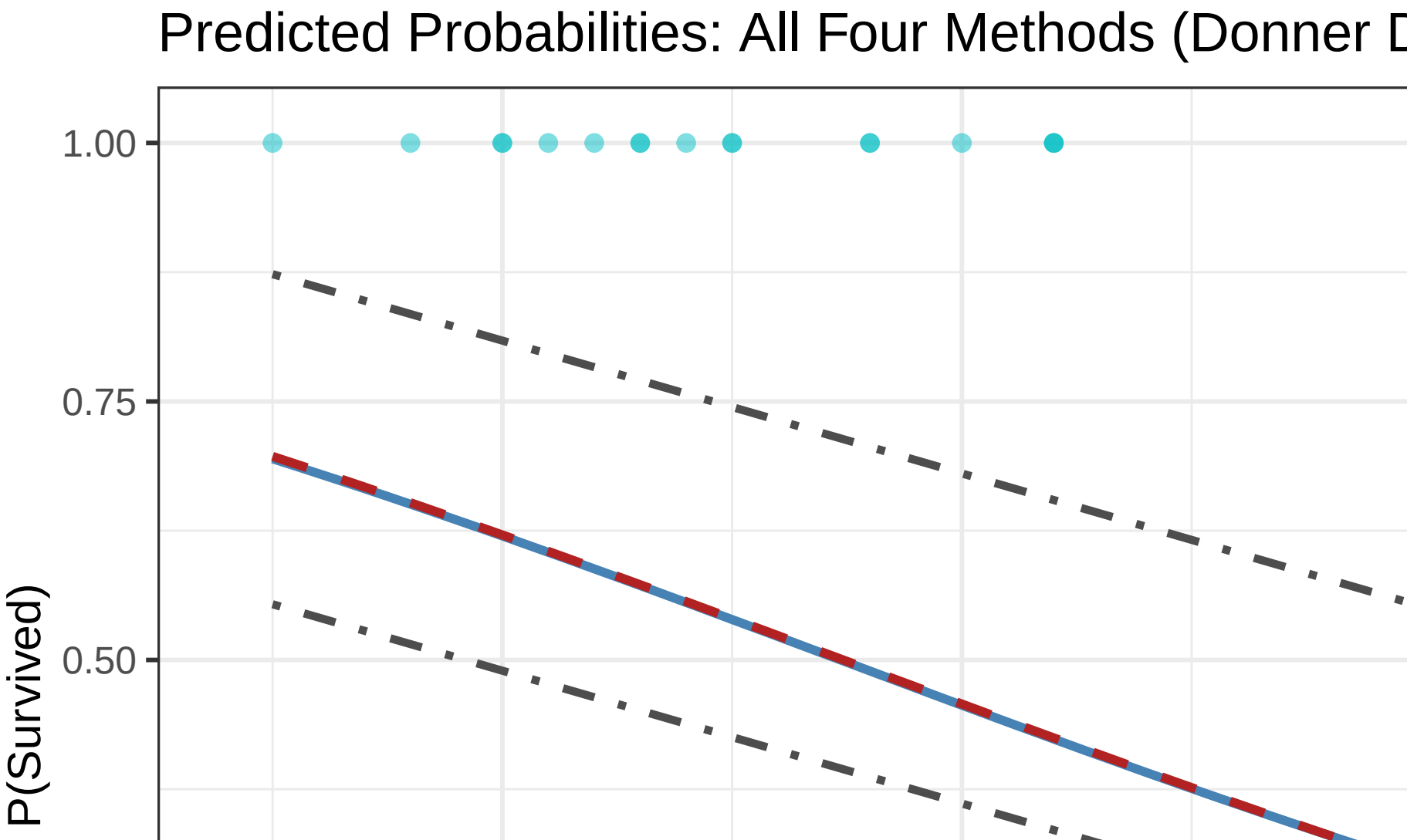
```r
round(as.matrix(coef(cv_lasso, s = "lambda.1se")), 5)
```

All Four Methods — Code

```
ggplot(donner, aes(x = Age)) +
  geom_point(aes(y = mystatus, color = Status), alpha = 0.5
  geom_line(aes(y = pi_lpm, group = male), color = "gray30"
            linewidth = 0.9, linetype = "dotdash") +
  geom_smooth(aes(y = mystatus), method = "glm",
              method.args = list(family = binomial(link = "
              se = FALSE, color = "steelblue", linewidth =
  geom_smooth(aes(y = mystatus), method = "glm",
              method.args = list(family = binomial(link = "
              se = FALSE, color = "firebrick", linewidth =
  labs(title = "Predicted Probabilities: All Four Methods (
       y = "P(Survived)", x = "Age",
       caption = "Blue = Logistic  |  Red = Probit  |  Gray
  theme_bw()
```

All Four Methods — Plot

```
## `geom_smooth()` using formula = 'y ~ x'
## `geom_smooth()` using formula = 'y ~ x'
```



Confusion Matrix Comparison — Code

```
threshold <- 0.5

results <- donner %>%
  mutate(
    pred_lpm    = factor(ifelse(pi_lpm    > threshold, 1, (
    pred_logit  = factor(ifelse(pi_logit  > threshold, 1, (
    pred_probit = factor(ifelse(pi_probit > threshold, 1, (
    pred_lasso  = factor(ifelse(
      predict(cv_lasso, newx = X_mat, s = "lambda.min",
              type = "response")[, 1] > threshold, 1, 0)),
    actual = factor(mystatus)
  )

acc <- function(pred, actual) round(mean(pred == actual), 4

cat(sprintf("%-20s %s\n", "Method",    "Accuracy"))
cat(sprintf("%-20s %s\n", "OLS (LPM)", acc(results$pred_lpm
cat(sprintf("%-20s %s\n", "Logistic",  acc(results$pred_log
```

Confusion Matrix Comparison — Output

```
## Method                    Accuracy

## OLS (LPM)                 0.7556

## Logistic                  0.7778

## Probit                    0.7778

## Lasso                     0.7556
```

Probit Confusion Matrix

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0  1
##          0 23  8
##          1  2 12
##
##                Accuracy : 0.7778
##                  95% CI : (0.6291, 0.888)
##     No Information Rate : 0.5556
##     P-Value [Acc > NIR] : 0.00166
##
##                   Kappa : 0.5361
##
##  Mcnemar's Test P-Value : 0.11385
##
##             Sensitivity : 0.6000
##             Specificity : 0.9200
```

Model Comparison Summary

Criterion	OLS (LPM)	Logistic	**Probit**	Lasso
Predictions in $[0, 1]$?	No	Yes	Yes	Yes
Link function	Identity	Logit	Φ^{-1} **(normal)**	Logit or Probit
Error distribution	Normal	Logistic	**Normal**	Logistic
Estimation method	OLS (closed form)	MLE (IRLS)	MLE (Fisher Scoring)	MLE + ℓ_1 penalty
Coefficient interpretation	$\Delta\hat{y}$	Log-odds / Odds ratio	Marginal effects (AME)	Log-odds (shrunk)
Variable selection?	No	No	No	**Yes**
Tuning required?	No	No	No	Yes (λ)

Key Takeaways

- **Linear regression (OLS)** is not appropriate for binary outcomes. Predicted values are not bounded to $[0, 1]$ and the model is misspecified.
- **Logistic and probit regression** produce nearly identical predicted probabilities and model fit. Coefficients differ in scale by approximately 1.6x due to differing error distributions, but substantive conclusions are the same.
- **Probit is preferred** in economics and political science due to the latent variable framework and field convention. Coefficients are interpreted via average marginal effects.
- **Logistic regression is preferred** when odds ratios are the natural effect measure.
- **Lasso** extends either model with an ℓ_1 penalty for automatic variable selection. Standard p-values and confidence intervals are not available after lasso selection.
- For government and political science applications, probit regression is the standard and appropriate choice.

Citations | References

1. **Abbott, M. G.** (n.d.). *ECON 452 Note 15: Marginal Effects in Probit Models*. Queen's University. https://econ.queensu.ca/faculty/abbott/econ452/452note15.pdf
2. **Friedman, J., Hastie, T., & Tibshirani, R.** (2010). Regularization Paths for Generalized Linear Models via Coordinate Descent. *Journal of Statistical Software*, 33(1). https://glmnet.stanford.edu
3. **Hanck, C., Arnold, M., Gerber, A., & Schmelzer, M.** (2020). *11.2 Probit and Logit Regression*. Introduction to Econometrics with R. https://www.econometrics-with-r.org/11.2-palr.html
4. **Lucero, C.** (2026). *Lecture 16: Classification & Generalized Linear Models*. CMDA 4654, Virginia Tech.
5. **MetricGate.** (n.d.). *Probit vs. Logit Models for Binary Outcomes*. https://metricgate.com/blogs/probit-vs-logit-regression/
6. **RStudio.** (n.d.). *RStudio Addins*. https://rstudio.github.io/rstudioaddins/
7. **Taboga, M.** (2021). *Probit classification model - Maximum likelihood*. Lectures on Probability Theory and Mathematical Statistics. https://www.statlect.com/fundamentals-of-statistics/probit-model-maximum-likelihood
8. **Urban Institute.** (n.d.). *Logit and Probit Regression*. https://www.urban.org/research/data-methods/data-analysis/quantitative-data-analysis/impact-analysis/regression-techniques/logit-and-probit-regression
9. **Wikimedia Commons.** (2021). *File:Hokkaido.parliamentary.svg*. https://commons.wikimedia.org/wiki/File:Hokkaido.parliamentary.svg
10. **Wikipedia contributors.** (2024). *Logistic regression*. Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/wiki/Logistic_regression
11. **Xie, Y.** (2024). *Xaringan: Presentation Ninja* [R package]. https://github.com/yihui/xaringan
12. **Yanai, Y.** (n.d.). *Logit and Probit Models*. Research Methods 1. https://yukiyanai.github.io/teaching/rm1/contents/R/logit-n-probit.html

- R Packages: `glmnet`, `caret`, `margins`, `Sleuth3`, `dplyr`, `ggplot2`

Appendix

- Probit model - Maximum Likelihood details
- Convergence tolerance pseudocode logic